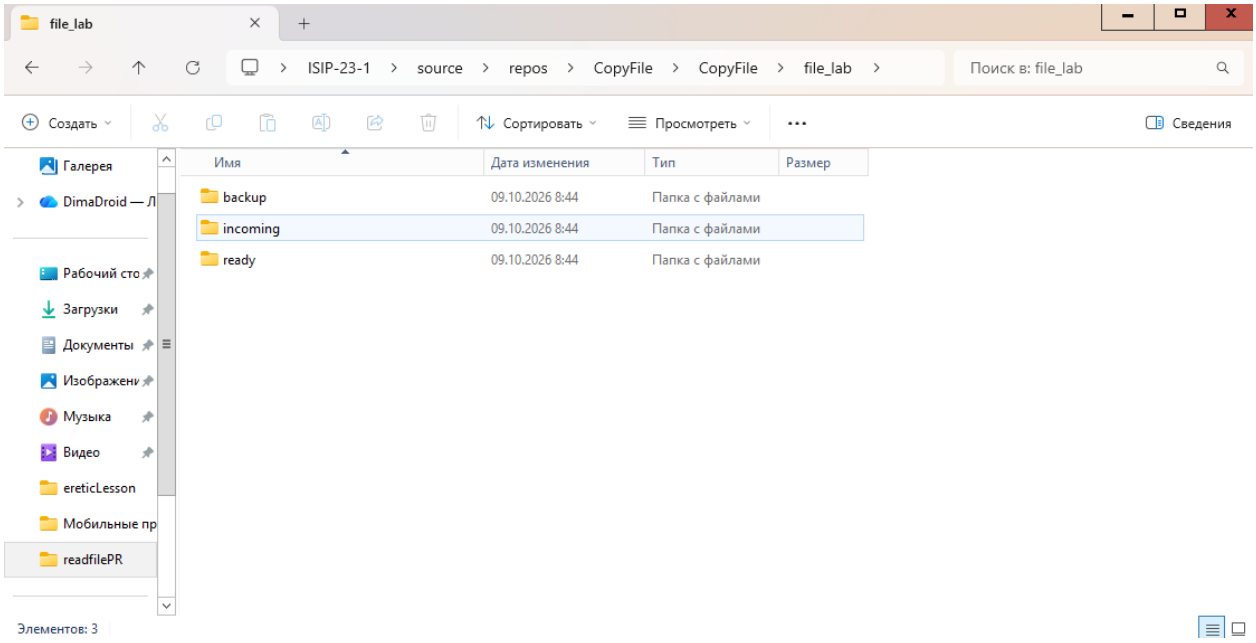
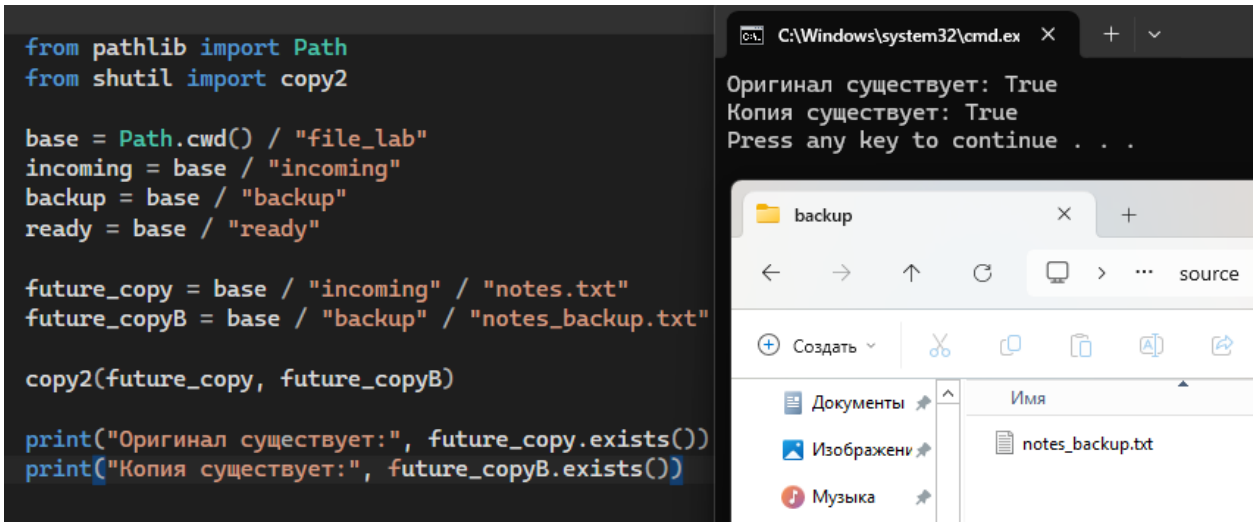
Копирование, перемещение и переименование

Исходная структура



Копирование



Переименование

```python
from pathlib import Path
from shutil import copy2

base = Path.cwd() / "file_lab"
incoming = base / "incoming"
backup = base / "backup"
ready = base / "ready"

future_copy = base / "incoming" / "data.txt"
future_copyB = base / "incoming" / "measurements.txt"

future_copy.rename(future_copyB)

print("Старое имя существует:", future_copy.exists())
print("Новое имя существует:", future_copyB.exists())
```

```
C:\Windows\system32\cmd.ex
Старое имя существует: False
Новое имя существует: True
Press any key to continue . . .
```

Старое имя выдает False из-за того, что exists () проверяет наличие файла по пути и в старом варианте имени такой путь не к чему не ведет

Перемещение

```python
from pathlib import Path
from shutil import copy2
from shutil import move

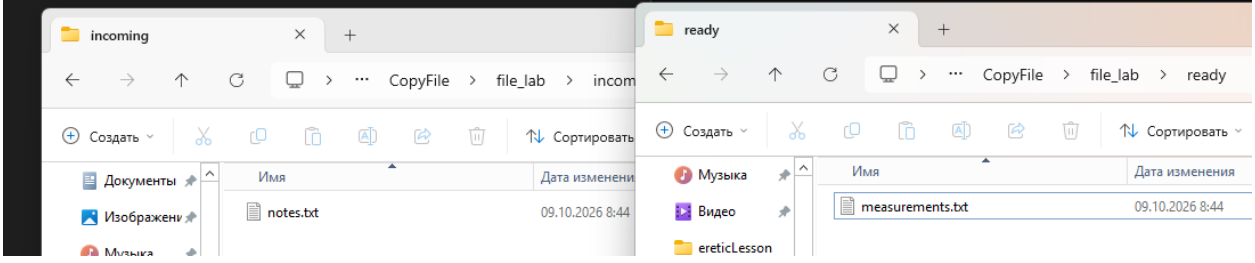
base = Path.cwd() / "file_lab"
incoming = base / "incoming"
backup = base / "backup"
ready = base / "ready"

future_copy = base / "incoming" / "measurements.txt"
future_copyB = base / "ready" / "measurements.txt"

move(future_copy, future_copyB)

print("В incoming остался файл:", future_copy.exists())
print("В ready появился файл:", future_copyB.exists())
```

```
C:\Windows\system32\cmd.ex
В incoming остался файл: False
В ready появился файл: True
Press any key to continue . . .
```



Код итоговой программы

```
from pathlib import Path
from shutil import copy2, move

base = Path.cwd() / "file_lab"
incoming = base / "incoming"
backup = base / "backup"
ready = base / "ready"

incoming.mkdir(parents=True, exist_ok=True)
backup.mkdir(parents=True, exist_ok=True)
ready.mkdir(parents=True, exist_ok=True)

with open(incoming / "task.txt", "w", encoding ="utf-8") as file:
        file.write("Учебный отчет\n")
        file.write("Статус: task\n")

with open(incoming / "config.txt", "w", encoding ="utf-8") as file:
        file.write("Учебный отчет\n")
        file.write("Статус: config\n")

with open(incoming / "draft.txt", "w", encoding ="utf-8") as file:
        file.write("Учебный отчет\n")
        file.write("Статус: draft\n")

orifinal = base / "incoming" / "task.txt"
future_copy = base / "backup" / "task_backup.txt"

if not future_copy.exists():
        copy2(orifinal, future_copy)
        print("Копия:", future_copy.exists())

original = base / "incoming" / "config.txt"
renamed = base / "incoming" / "settings.txt"

if original.exists() and not renamed.exists():
        original.rename(renamed)
        print("Старое имя:", original.exists())
        print("Новое имя:", renamed.exists())

mover = base / "ready" / "settings.txt"
if renamed.exists() and not mover.exists():
        move(renamed, mover)
        print("Файл в incoming:", mover.exists())
        print("Файл в ready:", mover.exists())

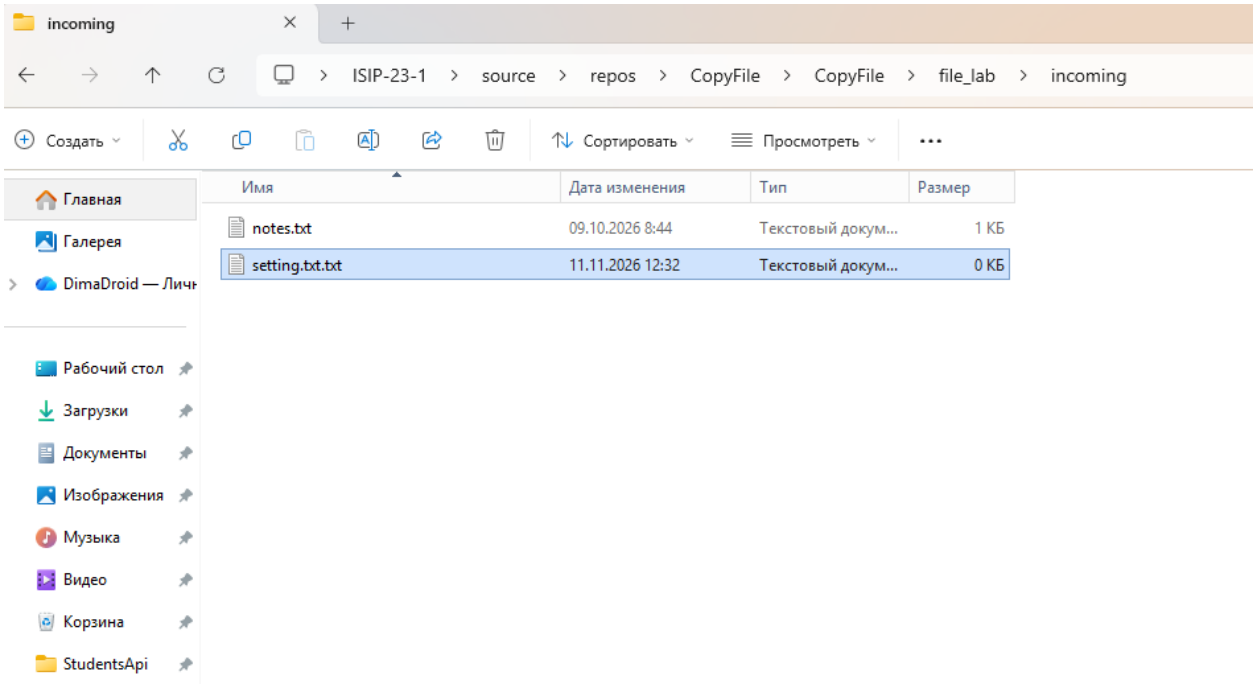
print("Содержимое incoming:")
for item in incoming.iterdir():
        print(item.name)
print("Содержимое backup:")
for item in backup.iterdir():
        print(item.name)
print("Содержимое ready:")
for item in ready.iterdir():
        print(item.name)
```

Самостоятельное изменение

```python
original = base / "incoming" / "config.txt"
renamed = base / "incoming" / "settings.txt"

if original.exists() and not renamed.exists():
    original.rename(renamed)
    print("Старое имя:", original.exists())
    print("Новое имя:", renamed.exists())
```

Создал файл config.txt в incoming и переименовал его в settings.txt



Контрольные вопросы

1. При копировании создаётся второй экземпляр файла с сохранением оригинала, а при перемещении файл переносится в другой каталог и остаётся в единственном экземпляре.
2. Исходный файл не удаляется и остаётся на своём прежнем месте.
3. После выполнения операции исходный путь перестаёт содержать перемещённый файл.
4. Метод применяется для изменения имени файла без создания его копии.
5. Проверка защищает данные от случайной перезаписи файла с таким же именем.
6. Ограничение рабочей папкой обеспечивает безопасность и предотвращает повреждение системных файлов или документов других пользователей.
7. С помощью вызова exists() подтверждают, что старый путь перестал существовать, а новый путь стал доступен.
8. Это происходит потому, что копирование создаёт дубликат содержимого по новому пути без удаления исходного объекта.

Вывод

В ходе лабораторной работы были освоены базовые операции с файловой структурой в Python с использованием библиотек pathlib и shutil. На практике было изучено различие между созданием дубликата через copy2(), изменением имени через rename() и смене каталога через move(). Проверка путей с помощью exists() и iterdir() наглядно подтвердила фактическое изменение файлов и показала важность защиты от случайной перезаписи.